

## الحلقات التكرارية (Loops) :

من الأدوات الأساسية التي تتيح تنفيذ أسطر معينة من الكود عدة مرات بناءً على شرط محدد، مما يوفر الوقت والجهد في كتابة الأكواد المتكررة بدلاً من كتابة نفس الأسطر مرات كثيرة وتتمثل الوظيفة الأساسية لهذه الحلقات في "إدارة التدفق المنطقي للبيانات"؛ حيث يقوم البرنامج باختبار شرط الاستمرار في بداية كل حلقة تكرار، فإذا تحقق الشرط نُفذت الأوامر، وإذا لم يتحقق خرج البرنامج من الحلقة التكرارية.

## الحلقات التكرارية في لغة PHP

• حلقة **for**

• حلقة **while**

• حلقة **do-while**

• حلقة **foreach**

حلقة التكرار المحددة (for loop)

لتنفيذ كتلة من الكود لعدد محدد من المرات. ويبرز دورها الوظيفي كأداة تضمن تكرار المهام التعليمية أو الإجرائية ضمن نطاق عددي ثابت، مما يجعلها بمثابة "نموذج للتكرار المنضبط" داخل البرنامج. وتعتمد آلية عملها على دمج ثلاثة أركان منطقية في سطر برمجي واحد وهي: (تحديد بداية التكرار، وضع شرط الاستمرار، وتحديد مقدار الزيادة)، حيث يختبر البرنامج الشرط في كل دورة؛ فإذا كان متحققاً، يُنفذ الكود البرمجي، ومع نهاية كل دورة يتم تحديث قيمة العداد آلياً.

ويستمر هذا التسلسل الإجرائي طالما ظل الشرط صائباً، أما في حال وصول العداد إلى النهاية المطلوبة (انكسار الشرط)، فإن البرنامج يتوقف عن التكرار فوراً ويتجاوز هذه الكتلة لينتقل إلى مرحلة "استكمال العمليات خارج حلقة التكرار". ويحقق هذا النمط كفاءة عالية في تدفق البيانات، حيث يضمن تنفيذ العمليات بالعدد المطلوب تماماً دون زيادة ترهق موارد النظام أو تداخل في المسارات الإجرائية، وهو ما ينعكس على سلاسة الأنظمة التفاعلية وقدرتها على معالجة البيانات الضخمة بدقة.

## المكونات البنائية لحلقة (for)

يتألف الهيكل البرمجي لحلقة (for) من خمسة عناصر أساسية تتكامل فيما بينها لإدارة عملية التكرار المنضبط:

1. **الأداة التكرارية: (for)** هي الكلمة المحجوزة التي تُعلن للمحرك البرمجي عن بدء حلقة تكرار محددة العناصر، وهي بمثابة المفتاح الذي يفتح مسار التكرار.

2. **محددات الحلقة: (Control Expressions)** تتمثل في القوسين الهلاليين ( ) اللذين يضمن بداخلها التعبيرات الثلاثة (بداية التكرار، شرط الاستمرار، ومقدار الزيادة) ويفصل بينهما الفاصلة المنقوطة .;

– **نقطة الانطلاق: (Initialization)** وتُستخدم لتحديد القيمة الابتدائية للمتغير المسؤول عن العد داخل الحلقة.

– **شرط الاستمرار: (Condition)** وهو الاختبار الذي يتم من خلاله فحص ما إذا كان يجب على الحلقة متابعة التكرار أم التوقف.

– **معامل التحديث: (Increment/Decrement)** ويقوم بتعديل قيمة متغير العد (بالزيادة أو النقصان) بعد الانتهاء من تنفيذ كل دورة تكرارية.

3. **نطاق التنفيذ: (Scope)** يتمثل في الأقواس المتعرجة { } التي تحدد حيز الأوامر التابعة للحلقة، وتضمن حصر كود التكرار بداخلها.

4. **الكتلة البرمجية: (Code to be executed)** هي مجموعة التعليمات والأوامر الموجودة داخل الأقواس المتعرجة، والتي يتم تنفيذها بشكل متتابع في كل دورة تكرارية طالما ظل الشرط صائباً.

## خطوات كتابة حلقة التكرار المحدد: (for loop)

1. **إعلان حلقة التكرار:** تبدأ الجملة بالكلمة المحجوزة for لإعلان عن بدء عملية تكرار منظمة تخضع لضوابط محددة مسبقاً.

2. صياغة محددات الحلقة :يتم فتح قوسين هلالين ( ) مباشرة بعد for لضبط مسار التكرار من خلال ثلاثة عناصر أساسية:

- تحديد نقطة الانطلاق: (Initialization) تبدأ بتعريف متغير يمثل "العداد" ويُسمى غالباً \$counter (أو أي اسم يعبر عن دوره)، حيث تُسند إليه القيمة الابتدائية التي سيبدأ منها البرنامج العد.

- وضع شرط الاستمرار: (Condition) يكتب بعد الفاصلة المنقوطة الأولى، وهو المعيار الذي يفحصه البرنامج قبل كل دورة تكرار؛ فإذا كان الشرط "متحققاً" استمر التكرار، وإذا أصبح "غير متحقق" توقف البرنامج فوراً.

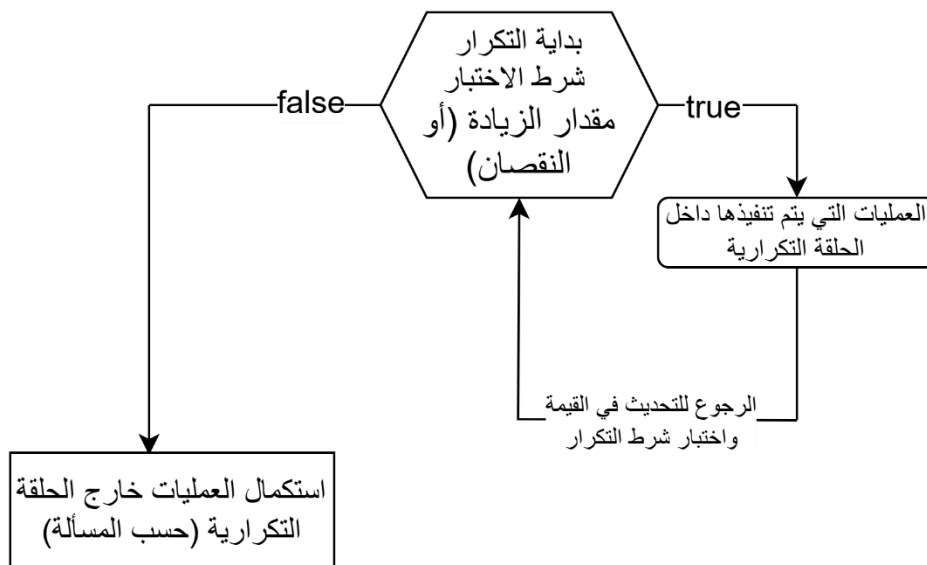
- تحديد معامل التحديث: (Increment/Decrement) يكتب بعد الفاصلة المنقوطة الثانية، وهو المسؤول عن تغيير قيمة العداد (بالزيادة أو النقصان) بعد نهاية كل دورة، لضمان انتقال الحلقة نحو هدفها ومنع الدخول في تكرار لا نهائي.

3. فتح نطاق التنفيذ: يتم وضع قوس متعرج مفتوح { للإشارة إلى بداية الكتلة البرمجية التي سيتم تكرار تنفيذها.

4. كتابة التعليمات البرمجية: يتم كتابة الأوامر المراد تنفيذها داخل النطاق، مع مراعاة إزاحتها جهة اليمين لضمان وضوح تبعيتها للحلقة التكرارية.

5. إغلاق النطاق: يتم إنهاء الحلقة بقوس متعرج مغلق } ليمثل نهاية مسار التكرار، وهو ما يوجه البرنامج للعودة إلى رأس الحلقة لتحديث العداد وفحص الشرط مرة أخرى.

```
( التحديث ; الشرط ; البداية )  
{  
الكود الذي سيتم تكراره طالما أن الشرط متحقق  
}
```



### آلية التنفيذ المنطقية لحلقة: (for)

يعتمد التكرار بشكل أساسي على تحقق شرط الاختبار:

- **في حالة الصواب: (True)** إذا وجد المحرك البرمجي أن نتيجة "شرط الاختبار" صائبة، فإنه يسمح بالدخول لتنفيذ "العمليات داخل الحلقة التكرارية". وبمجرد الانتهاء منها، ينتقل البرنامج تلقائياً لتنفيذ "مقدار الزيادة أو النقصان" لتحديث قيمة العداد، ثم يعود مرة أخرى إلى نقطة فحص الشرط لبدء دورة جديدة.
- **في حالة الخطأ: (False)** إذا لم يتحقق الشرط (أي أصبحت النتيجة خاطئة)، فإن البرنامج يتوقف تماماً عن تكرار العمليات ويتجاوز مسار الحلقة، لينتقل مباشرة إلى مرحلة "استكمال العمليات خارج الحلقة التكرارية" لمتابعة تنفيذ الأسطر البرمجية اللاحقة.

```

for ($counter = 1; $counter <= 5; $counter += 1)
{
    echo $counter ;
}
  
```

```
for ($counter = 1; $counter <= 5; $counter += 1)
{
    echo $counter ;
}
```

## حلقة التكرار المشروط (while loop)

تُصنف حلقة (while) كأحدى أدوات "التحقق المسبق" في تدفق البرنامج، تماماً كحلقة (for)، حيث تتركز وظيفتها في تنفيذ مجموعة من الأوامر طالما ظل هناك شرط منطقي معين قائماً ومتحققاً. ويبرز دورها الوظيفي كأداة مرنة تتيح تكرار المهام التعليمية التي لا نعرف عدد مراتها بدقة مسبقاً، بل نربط استمرارها ببقاء الحالة متحققة، مما يجعلها بمثابة "نموذج للتكرار المرتبط بشرط".

وتعتمد آلية عملها على اختبار "شرط الاستمرار" في بداية كل دورة تكرارية؛ فإذا وجد البرنامج أن الشرط صائب (متحقق)، فإنه ينفذ الكتلة البرمجية الموجودة داخل الحلقة. ويستمر هذا التسلسل الإجرائي في حالة دوران مستمر طالما ظل الشرط متحققاً، ومع كل دورة يعيد البرنامج فحص الشرط من جديد قبل البدء في التنفيذ.

أما في حال تغيرت المعطيات وأصبح الشرط غير متحقق، فإن البرنامج يتوقف عن التكرار فوراً ويتجاوز هذه الكتلة لينتقل إلى مرحلة "استكمال العمليات خارج حلقة التكرار". ويحقق هذا النمط كفاءة عالية في التعامل مع البيانات الديناميكية، حيث يضمن استجابة النظام للمتغيرات اللحظية دون الدخول في تكرارات زائدة.

## المكونات البنائية لحلقة (while)

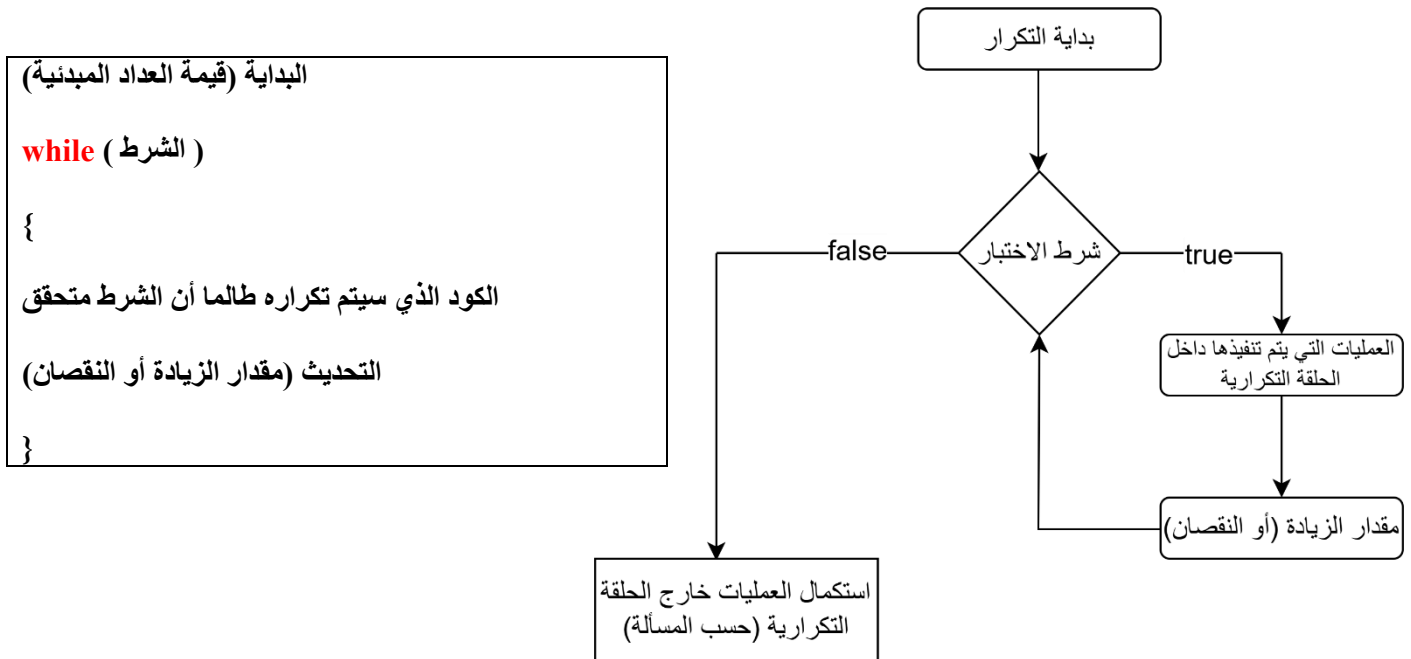
يتألف الهيكل البرمجي المتكامل لحلقة (while) من العناصر التالية التي تضمن انضباط عملية التكرار:

1. **بداية التكرار (القيمة المبدئية):** هي الخطوة التي تسبق الحلقة مباشرة، حيث يتم تعريف متغير (العداد) ومنحه القيمة الأولية التي سيعتمد عليها "شرط الاستمرار" في أول فحص له.

2. الأداة التكرارية: (while) هي الكلمة المحجوزة التي تُعلن عن بدء الحلقة، وتعمل بمبدأ "التحقق المسبق"؛ حيث لا تسمح بمرور التنفيذ إلا بعد فحص الحالة المنطقية للشرط.
3. شرط الاستمرار: (Condition) هو التعبير المنطقي الذي يُكتب بين قوسين هلاليين ( )؛ ويمثل المعيار الدائم الذي يقرر البرنامج بناءً عليه: هل يستمر التكرار (إذا كان صائباً) أم يتوقف (إذا كان خاطئاً).
4. نطاق التنفيذ: (Scope) يتمثل في الأقواس المتعرجة { } التي تحدد حيز الأوامر التابعة للحلقة، وتضمن عزل كود التكرار عن بقية أسطر البرنامج.
5. الكتلة البرمجية: (Code) هي التعليمات الموجودة داخل الأقواس، وتتضمن المهام والعمليات الأساسية المطلوب تكرار أدائها.
6. مقدار الزيادة أو النقصان (تحديث الحالة): هو السطر البرمجي الذي يغير قيمة العداد داخل الحلقة بعد كل دورة؛ لضمان وصول البرنامج لنقطة نهاية منطقية ومنع الدخول في تكرار لانتهائي.

#### خطوات كتابة حلقة التكرار المشروط: (while loop)

1. تحديد بداية التكرار (القيمة المبدئية): تبدأ العملية قبل كتابة الحلقة نفسها، حيث يتم تعريف متغير العداد مثل (\$counter) وإسناد القيمة الأولية له، وهي القيمة التي سيعتمد عليها البرنامج في أول عملية فحص للشرط.
2. إعلان حلقة التكرار: تبدأ الجملة بالكلمة المحجوزة while للإعلان عن بدء عملية تكرار مرهونة ببقاء حالة منطقية معينة، وتعمل وفق مبدأ "التحقق المسبق" قبل الدخول في أي دورة.
3. تحديد شرط الاستمرار: يتم فتح قوسين هلاليين ( ) مباشرة بعد while لكتابة المعيار المنطقي المراد فحصه؛ حيث يمثل هذا الجزء "بوابة العبور" التي يتحدد بناءً عليها استمرار التكرار أو توقفه.
4. فتح نطاق التنفيذ: يتم وضع قوس متعرج مفتوح { للإشارة إلى بداية الكتلة البرمجية التي سيتم تنفيذها طالما ظل الشرط "متحققاً".
5. كتابة التعليمات ومقدار التحديث: يتم كتابة الأوامر المراد تكرارها داخل النطاق، مع ضرورة كتابة سطر "تحديث العداد" (بالزيادة أو النقصان) كخطوة أخيرة داخل هذه الكتلة، لضمان تغير قيمة العداد ومنع الدخول في تكرار لا نهائي.
6. إغلاق النطاق: يتم إنهاء الحلقة بقوس متعرج مغلق } للإشارة إلى نهاية مسار التكرار، وهو ما يوجه البرنامج للعودة إلى رأس الحلقة لإعادة فحص الشرط بناءً على القيمة الجديدة للعداد.



### آلية التنفيذ المنطقية لحلقة: (while)

يعتمد استمرار التكرار في هذه الحلقة على نتيجة فحص "التحقق المسبق" للشرط:

- **في حالة الصواب: (True)** إذا وجد المحرك البرمجي أن نتيجة "شرط الاختبار" صائبة، فإنه يسمح بالدخول لتنفيذ "العمليات داخل الحلقة التكرارية". وعند الوصول لنهاية هذه العمليات، يتم تنفيذ سطر التحديث (تغيير قيمة المتغير)، ثم يعود البرنامج تلقائياً إلى نقطة فحص الشرط من جديد لبدء دورة تكرارية أخرى.
- **في حالة الخطأ: (False)** إذا وجد البرنامج عند نقطة الفحص أن الشرط غير متحقق، فإنه يتوقف تماماً عن تنفيذ أي عمليات داخل الحلقة، ويتجاوز هذا المسار بالكامل لينتقل مباشرة إلى مرحلة "استكمال العمليات خارج الحلقة التكرارية" لمتابعة تنفيذ بقية أسطر البرنامج.

```
$counter = 5;
while ($counter >= 1)
{
    echo $counter ;
    $counter--;
}
```

```
$counter = 5;

while ($counter >= 1)

{
    echo $counter ;

    $counter--;
}
```

### حلقة التكرار المشروط (do while loop)

تُصنف حلقة (do while) كأحدى أدوات "التحقق اللاحق" في تدفق البرنامج؛ حيث تختلف جذرياً عن حلقتي for و while في ترتيب الأولويات الإجرائية. وتتركز وظيفتها في ضمان تنفيذ مجموعة من الأوامر مرة واحدة على الأقل، بغض النظر عن حالة الشرط عند البداية، مما يجعلها بمثابة "نموذج للتكرار المعتمد على التنفيذ أولاً".

وتعتمد آلية عملها على البدء بتنفيذ الكتلة البرمجية مباشرة، ثم يليه اختبار "شرط الاستمرار" في نهاية الدورة؛ فإذا وجد البرنامج أن الشرط صائب (متحقق)، فإنه يعود لتكرار العملية مرة أخرى. ويستمر هذا التسلسل الإجرائي طالما ظل الشرط متحققاً، ولكن الفارق الجوهرى هو أن فحص الشرط يتم دائماً بعد انتهاء التنفيذ وليس قبله.

أما في حال كان الشرط غير متحقق منذ البداية، فإن البرنامج ينفذ الكود لمرة واحدة فقط ثم يتجاوز الحلقة لينتقل إلى مرحلة "استكمال العمليات خارج حلقة التكرار". ويحقق هذا النمط كفاءة عالية في المواقف التي تتطلب تفاعلاً أولاً مع المستخدم (مثل طلب إدخال بيانات) قبل تقييم مدى الحاجة لتكرار العملية.

## المكونات البنائية لحلقة (do while)

يتألف الهيكل البرمجي لحلقة (do while) من العناصر التالية التي تضمن استمرارية التنفيذ المنضبط:

1. **بداية التكرار (القيمة المبدئية):** الخطوة التي تسبق الحلقة لتعريف متغير العداد ومنحه القيمة الأولية التي سيتم تعديلها لاحقاً.

2. **أداة التنفيذ (do):** هي الكلمة المحجوزة التي تعطي الأمر بالبداية الفوري في التنفيذ دون انتظار فحص أي شروط.

3. **نطاق التنفيذ (Scope):** يتمثل في الأقواس المتعرجة { } التي تحصر بداخلها المهام المراد تنفيذها مرة واحدة على الأقل.

4. **الكتلة البرمجية (Code):** التعليمات الموجودة داخل الأقواس، وتتضمن المهام الأساسية بالإضافة إلى "مقدار التحديث" لتغيير حالة العداد.

5. **أداة التحقق (while):** تأتي في نهاية الهيكل لتعلن عن مرحلة فحص الشرط بعد إتمام التنفيذ.

6. **شرط الاستمرار (Condition):** التعبير المنطقي الذي يُكتب بين قوسين ( ) متبوعاً بفاصلة منقوطة ؛ وهو المعيار الذي يقرر هل يعود البرنامج للدوران أم يتوقف.

## خطوات كتابة حلقة التكرار المشروط: (do while loop)

1. **تحديد بداية التكرار (القيمة المبدئية):** تعريف متغير العداد (مثل \$counter وإسناد قيمة أولية له قبل الدخول في بنية الحلقة.

2. **إعلان بدء التنفيذ:** كتابة الكلمة المحجوزة do لفتح مسار التنفيذ الإجباري للدورة الأولى.

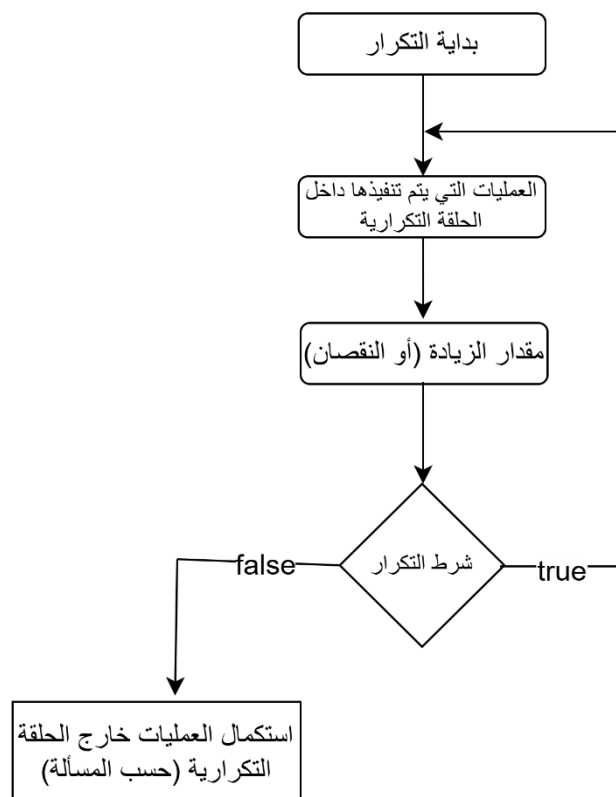
3. **فتح نطاق التنفيذ:** وضع قوس متعرج مفتوح { لبدء كتابة المهام التي ستنفذ قبل فحص الشرط.

4. **كتابة التعليمات ومقدار التحديث:** تدوين الأوامر المراد تنفيذها، مع ضرورة إنهاء الكتلة بسطر "تحديث العداد" (بالزيادة أو النقصان) لتعديل الحالة قبل الفحص.

5. إغلاق نطاق التنفيذ: وضع القوس المتعرج المغلق } للإشارة إلى انتهاء المهمة الحالية.

6. تحديد شرط الاستمرار: كتابة الكلمة المحجوزة while متبوعة بالشرط بين قوسين ( ) ثم الفاصلة المنقوطة ؛ ليوّجه البرنامج إما للعودة إلى do أو الخروج لاستكمال العمليات اللاحقة.

```
البداية (قيمة العداد المبدئية)
do
{
الكود الذي سيتم تنفيذه (مرة واحدة على الأقل)
التحديث (مقدار الزيادة أو النقصان)
}
while (الشرط);
```



### آلية التنفيذ المنطقية لحلقة: (do while)

يعتمد استمرار التكرار في هذه الحلقة على نتيجة فحص "التحقق اللاحق" للشرط بعد إتمام التنفيذ:

- في حالة الصواب (True) يبدأ المحرك البرمجي بالدخول مباشرة لتنفيذ "العمليات داخل الحلقة التكرارية" دون فحص الشرط في الدورة الأولى، وبعد الانتهاء منها يتم تنفيذ سطر التحديث (تغيير قيمة المتغير). ثم ينتقل البرنامج لنقطة فحص الشرط؛ فإذا وجدها صائبة، يعود تلقائياً للأعلى لبدء دورة تكرارية جديدة.

- **في حالة الخطأ (False):** إذا وجد البرنامج عند نقطة الفحص (التي تقع في نهاية الحلقة) أن الشرط غير متحقق، فإنه يتوقف عن العودة للأعلى ويتجاوز مسار التكرار، لينتقل مباشرة إلى مرحلة "استكمال العمليات خارج الحلقة التكرارية" لمتابعة تنفيذ بقية أسطر البرنامج. (يُلاحظ هنا أن العمليات قد نُفذت بالفعل لمرة واحدة على الأقل قبل الوصول لهذا الفحص).

```
$counter = 5;

do
{
    echo $counter;

    $counter--;
}

while ($counter >= 1);
```

```
$counter = 5;

do
{
    echo $counter;

    $counter--;
}

while ($counter >= 1);
```

### حلقة تكرار المصفوفات (foreach loop)

تُصنف حلقة (foreach) كأداة تكرار متخصصة ومصممة خصيصاً للتعامل مع "المصفوفات (Arrays)" والمجموعات البرمجية؛ حيث تختلف في جوهرها عن الحلقات السابقة (for, while) في كونها لا تعتمد على "شرط عددي" أو "شرط منطقي" خارجي، بل تعتمد وظيفتها على "عدد العناصر الموجودة" فعلياً داخل المصفوفة، مما يجعلها بمثابة "نموذج للتكرار المعتمد على البيانات".

وتعتمد آلية عملها على استخراج عناصر المصفوفة واحداً تلو الآخر بشكل تلقائي؛ ففي بداية كل دورة، يقوم البرنامج بسحب "قيمة" العنصر الحالي من المصفوفة وتخزينها في متغير مؤقت، ثم ينفذ الكتلة البرمجية بناءً على هذه القيمة. ويستمر هذا التسلسل الإجرائي بشكل آلي حتى يمر البرنامج على جميع عناصر المصفوفة من بدايتها إلى نهايتها، دون الحاجة لتدخل المبرمج في عملية تحديث العداد أو فحص النهاية.

أما في حال كانت المصفوفة فارغة (لا تحتوي على عناصر)، فإن البرنامج يتجاوز الحلقة تماماً منذ البداية لينتقل إلى مرحلة "استكمال العمليات خارج حلقة التكرار". ويحقق هذا النمط كفاءة واستقراراً عالياً في التعامل مع قواعد البيانات والقوائم التعليمية المتغيرة، حيث يضمن الوصول لكل عنصر بدقة وسهولة دون مخاطر الدخول في تكرارات لانهائية أو تجاوز حدود المصفوفة.

### المكونات البنائية لحلقة (foreach)

يتألف الهيكل البرمجي لحلقة (foreach) من العناصر التالية التي تضمن انسيابية التعامل مع المصفوفات:

1. **المصفوفة المستهدفة: (Array)** هي مصدر البيانات الذي يحتوي على مجموعة العناصر المراد معالجتها وتكرر الأوامر عليها.

2. **الأداة التكرارية: (foreach)** هي الكلمة المحجوزة التي تُعلن عن بدء دورة تكرار متخصصة تمر على كافة عناصر المصفوفة بشكل متسلسل.

3. **محددات الحلقة: (Control Expressions)** تتمثل في القوسين الهلاليين ( ) اللذين يضمنان بداخلها المصفوفة الأساسية والمتغير الذي سيمثل كل عنصر (باستخدام كلمة as).

4. **متغير العنصر الحالي: (Value)** هو متغير مؤقت يُعرف داخل القوسين، وتُخزن فيه قيمة العنصر الذي عليه الدور في كل دورة تكرارية.

5. **نطاق التنفيذ: (Scope)** يتمثل في الأقواس المتعرجة { } التي تحدد حيز الأوامر المطلوب تنفيذها على كل عنصر من عناصر المصفوفة.

6. **الكتلة البرمجية: (Code)** هي التعليمات التي تستخدم "قيمة العنصر الحالي" لأداء مهمة معينة (مثل طباعة الاسم أو معالجة الدرجة).

### خطوات كتابة حلقة تكرار المصفوفات: (foreach loop)

1. **تجهيز المصفوفة (مصدر البيانات):** تبدأ العملية بالتأكد من وجود مصفوفة تحتوي على مجموعة من القيم (مثل أسماء الطلاب أو الدرجات)، حيث يعتمد التكرار كلياً على وجود هذه البيانات.

2. إعلان حلقة التكرار: تبدأ الجملة بالكلمة المحجوزة `foreach` لإعلان عن بدء عملية تكرار ذكية تمر على عناصر المصفوفة عنصراً تلو الآخر.

3. تحديد متغيرات التحكم: يتم فتح قوسين هلاليين ( ) لكتابة اسم المصفوفة الأساسية متبوعاً بالكلمة المحجوزة `as` ثم تسمية متغير مؤقت (مثل `$item`) ليمثل قيمة العنصر الحالي في كل دورة.

4. فتح نطاق التنفيذ: يتم وضع قوس متعرج مفتوح { للإشارة إلى بداية الكتلة البرمجية التي سيتم تنفيذها لكل عنصر من عناصر المصفوفة.

5. كتابة التعليمات البرمجية: يتم تدوين الأوامر المراد تنفيذها داخل النطاق، مع استخدام "متغير العنصر الحالي" لمعالجة البيانات أو عرضها، وهنا لا يحتاج المبرمج لكتابة سطر "تحديث العداد" لأن الانتقال للعنصر التالي يتم تلقائياً.

6. إغلاق النطاق: يتم إنهاء الحلقة بقوس متعرج مغلق } للإشارة إلى نهاية مسار التكرار للعنصر الحالي، مما يوجه البرنامج للعودة وسحب العنصر التالي من المصفوفة حتى انتهاء كافة العناصر.

```
( القيمة $ as المصفوفة $ )  
foreach  
  
{  
  
الكود الذي سيتم تكراره لكل عنصر موجود داخل المصفوفة //  
  
}
```

آلية التنفيذ المنطقية لحلقة: `(foreach)`

يعتمد استمرار التكرار في هذه الحلقة على "توافر العناصر" داخل المصفوفة:

- في حالة وجود عناصر: **(Elements Exist)** يقوم المحرك البرمجي بالإمساك بأول عنصر في المصفوفة ووضعه في المتغير المؤقت، ثم يسمح بالدخول لتنفيذ "العمليات داخل الحلقة التكرارية". وبعد الانتهاء، ينتقل البرنامج تلقائياً للعنصر التالي ويكرر العملية، وهكذا دواليك حتى استنفاد كافة العناصر.

- في حالة انتهاء العناصر (Empty/Finish) بمجرد وصول البرنامج لنهاية المصفوفة، أو إذا كانت المصفوفة فارغة أصلاً، فإنه يتوقف عن الدوران فوراً ويتجاوز مسار الحلقة، لينتقل مباشرة إلى مرحلة "استكمال العمليات خارج الحلقة التكرارية".

```
$arr = array (10, 20, 30, 40, 50, 60);
```

```
foreach ($arr as $val)
```

```
{
```

```
    echo $val . " ";
```

```
}
```

```
$arr = array (10, 20, 30, 40, 50, 60);
```

```
foreach ($arr as $val)
```

```
{
```

```
    echo $val . " ";
```

```
}
```

```
$ages = array
```

```
(
```

```
    "Anjali" => 25,
```

```
    "Kriti" => 30,
```

```
    "Ayushi" => 22
```

```
);
```

```
foreach ($ages as $name => $age)
```

```
{
```

```
    echo $name . " => " . $age . "\n";
```

```
}
```

```
$ages = array
```

```
(
```

```
    "Anjali" => 25,
```

```
    "Kriti" => 30,
```

```
    "Ayushi" => 22
```

```
);
```

```
foreach ($ages as $name => $age)
```

```
{
```

```
    echo $name . " => " . $age . "\n";
```

```
}
```